Introduction to Processor Architecture

Project Phase II: Pipelined RISC-V Processor Design

Spring 2026

Due Date: 9 March 2026

1 Introduction

Congratulations on successfully implementing the Sequential version of the RISC-V processor. While the sequential design is functional, it suffers from low throughput as instructions are executed one at a time. In this phase, you will implement a **5-stage Pipelined Processor** to improve instruction throughput.

This project builds directly upon your previous sequential implementation. You are required to implement proper pipeline registers, hazard detection, data forwarding, and control hazard handling.

Your pipelined processor must support the same subset of the RISC-V ISA as the sequential design. Ensure the following instructions are fully functional:

2 The 5-Stage Pipeline

You must decompose your sequential datapath into five distinct stages. You are required to instantiate pipeline registers between these stages to store data and control signals.

2.1 Pipeline Stages

1. **Instruction Fetch (IF):** Fetches the instruction from Instruction Memory and increments the PC.
2. **Instruction Decode (ID):** Decodes the instruction, reads from the Register File, and generates Immediate values.
3. **Execute (EX):** Performs arithmetic operations (ALU) and calculates branch targets.
4. **Memory (MEM):** Accesses Data Memory for load/store instructions.
5. **Write Back (WB):** Writes results back to the Register File.

2.2 Pipeline Registers

You must implement the following pipeline registers. Ensure that **all** necessary control signals (e.g., `RegWrite`, `MemRead`, `MemToReg`) propagate through these registers alongside the data. The register files are IF/ID Registers, ID/EX Registers, EX/MEM Registers and MEM/WB Registers.

3 Hazard Handling

Pipelining introduces dependencies that must be resolved to ensure correct execution. You must implement the following units to handle Data and Control hazards.

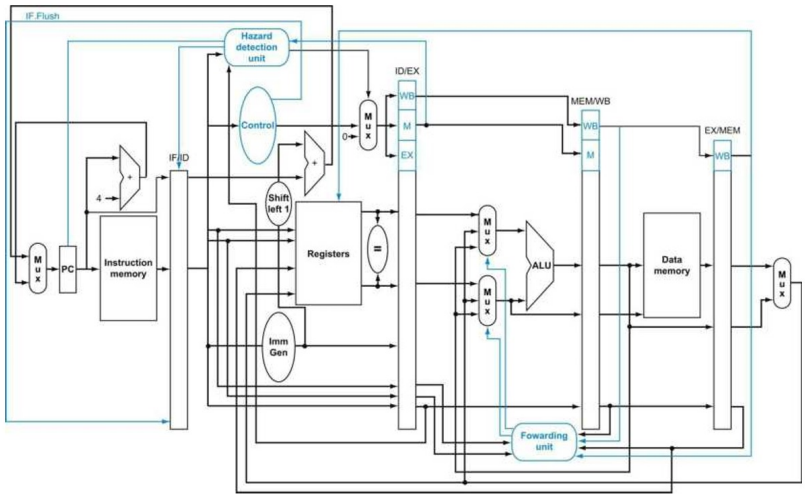

FIGURE 4.62 The final datapath and control for this chapter.

Figure 1: Reference from the textbook. Please note that this image may not contain all the connections and signals

3.1 Data Forwarding Unit

The sequential flow assumes data is written before it is read. In a pipeline, an instruction in the EX stage may need data that is currently in the MEM or WB stage. You must implement a **Forwarding Unit** to bypass data from the pipeline registers to the ALU inputs. Implement:

1. EX Hazard (MEM to EX)
2. MEM Hazard (WB to EX)

Note: Ensure that you do not forward if the destination register is $x0$ or if `RegWrite` *is not asserted. Also, there may be other cases of data forwarding. Figuring them out and handling these will be counted as bonus.*

3.2 Hazard Detection Unit

Certain hazards cannot be resolved by forwarding alone (Why?). We want you to handle them appropriately.

Hint: Detect when an instruction in ID depends on a load instruction in EX and stall the pipeline.

3.3 Control Hazards

For this project, assume a **static branch prediction: always NOT TAKEN**.

- Continue fetching instructions sequentially ($PC + 4$).
- If the branch is actually taken (resolved in the EX stage), you must **Flush** the pipeline.

4 Report

The report must include specific test cases and waveforms with relevant signals plotted showcasing different cases of forwarding and other hazard handling mechanisms. Make sure to explain all the mechanisms to handle hazards that have been used in your project. At the end of the report, please include a brief section mentioning the contribution of each member of the team.

The submission and testing format will be similar to the sequential processor. We will be testing your code with our `instructions.txt`. After executing `iverilog pipe_tb.v` (ensure that this file is present which reads & generates the required files), we will compare the generated `register_file.txt` with the golden reference we have. Register values and clock cycle count will be checked, so ensure that no clock cycles are wasted in performing other additional tasks. A sample input and output has been uploaded. Note that after the last valid instruction (call it `I`, 4 dummy instructions are added to the testbench (like `add x0, x0, x0`) to ensure that the instruction `I` can write back to the register file (if required).

5 Bonus Features (Optional)

Implementing advanced features will earn bonus marks (capped to 30 marks). Ideas include:

- Any additional case of forwarding that is handled.
- Improved Branch Prediction: Changing the hardware to reduce the branch misprediction penalty.
- More instructions or techniques to extract performance.

Please note that it is perfectly possible to score full without implementing any bonus sections. However, implementing them will be a great learning experience.

6 Grading Breakdown (Total: 30 Marks)

Component	**Marks**
Report	5
Automated Test cases	5
Viva	20
Bonus (optional)	5